

OPEN-SOURCE LEARNING PLATFORM · MIT

daig

DevOps Intern Bootcamp

Break it. Diagnose it. Ship it safely.

A hands-on foundation & track for Cloud · DevOps · SRE · DevSecOps · Platform Engineering

● 3-tier app

● Kubernetes

● Terraform ×3 clouds

● Observability + SLOs

● Vault · DevSecOps

● github.com/adeelarshad414/devops-interns-proj

Instructor-led kickoff · 6-day rotation

Why we learn by breaking things

Most people “learn DevOps” by watching. Then the first 2 a.m. incident hits — and none of it transfers.

THE GAP

- Tutorials teach you to copy commands — not to diagnose a failure.
- You were never allowed to break production.
- You never read real signals: logs, metrics, traces, profiles.
- So the knowledge doesn’t survive contact with a real incident.

THE ANSWER: DAIG

- A real, instrumented platform that is broken on purpose.
- Every fault has a symptom, a signal, and a one-command revert.
- You practice the loop until it’s muscle memory.

THE DRIVING PROBLEM

The iftar spike

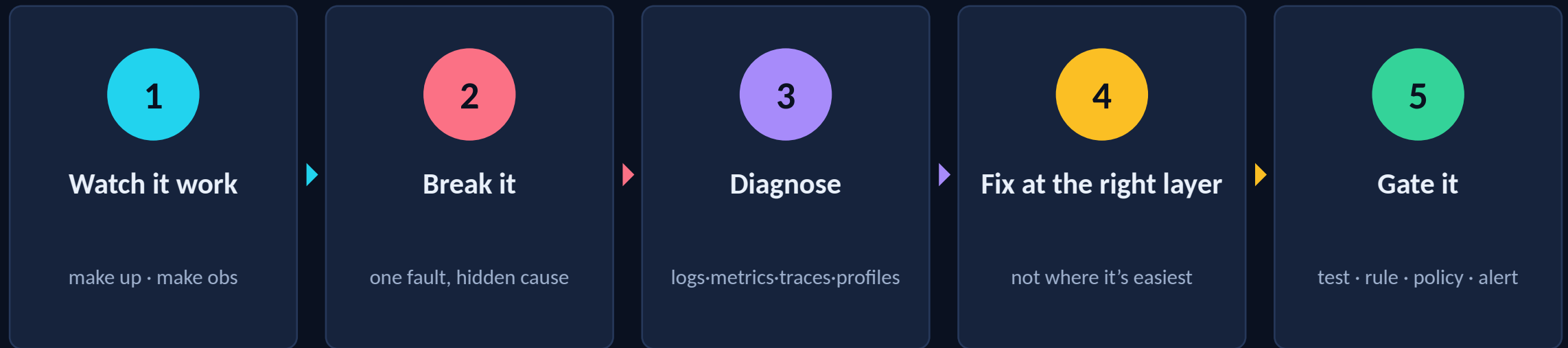
40 minutes before sunset in Ramadan, an entire country orders dinner in the same second.



This one problem drives every lesson: scaling, latency, back-pressure, SLOs, incident diagnosis.

The loop — repeated at every layer

You run this same five-step loop each phase, one layer deeper: Linux → containers → k8s → pipelines → security.



Step 5 is the one everyone skips — and the one that matters. A fix without a gate regresses within two sprints. Every bug you close leaves behind a test, a rule, a policy, or an alert.

What is daig — the system you operate

A three-tier food-delivery platform: instrumented end-to-end, with every boundary enforced, not documented.

Functional + Technical Solution

One browser request, three tiers, four observability pillars, secrets from a vault — every boundary enforced, not documented.



One browser request → web → orders → kitchen → dispatch → Postgres/Redis, watched by four observability pillars, with secrets from a vault.

The track — a 7-phase capstone

Each phase is the loop at a new layer. You leave evidence + a gate at every step.

The 7-Phase Capstone Roadmap

Each phase repeats the loop at a new layer: watch → break → diagnose → fix → gate. Evidence required at every step.



Do them in order — each builds on the last. Full brief in `ASSIGNMENT.md`; worked answers (instructor-held) in `SOLUTION.md`.

Day 0 — get your hands dirty

Ten minutes to a running, verified stack. No cloud account needed.

PREREQUISITES

- Docker Desktop (or Docker Engine) + `make`
- Node 22 (via nvm — the repo pins it in .nvmrc)
- git, and a terminal you're comfortable in
- Admin rights on your laptop (containers need it)

FIRST RUN

```
git clone <repo> && cd devops-interns-proj
cp .env.example .env
make hooks # install the pre-commit gate
make up && make seed && make smoke
```

PROVE IT WORKS

```
# every tier answers
make smoke
# place a real order end-to-end
./scripts/integration-order.sh
```

THE EXIT-78 EXERCISE

```
make broken
docker run --rm tkxel/daig-orders:broken
# reads: "Missing DATABASE_URL", exits 78
```

Read the failure the way the platform intends — a named missing variable, not a stack trace.

The foundation stack

Nine areas, one system. Each has concepts, a cheatsheet, and something to break — all in TRAINING.md.

1 Linux & the runtime

signals · exit codes · systemd · sockets

2 Containers & Docker

images · multi-stage · non-root · healthchecks

3 Compose & the 3 tiers

health-gated startup · overlays · volumes

4 Kubernetes

deploy · scale · self-heal · roll back

5 IaC — Terraform & Ansible

state · providers · idempotency · 3 clouds

6 CI/CD

prove → gate → ship, or refuse

7 DevSecOps & Secrets

7 gates · SBOM · signing · Vault

8 Observability & SRE

RED/USE · SLOs · burn-rate · the pivot

9 Chaos engineering

steady state · blast radius · revert

Containers & Docker

An image is an immutable, layered filesystem. The craft: small, reproducible, non-root, observable.

FOUNDATION

- Multi-stage builds → a lean runtime image = smaller attack surface.
- Reproducible: build from a lockfile (npm ci), never npm install.
- Least privilege: a non-root USER, dropped caps, a HEALTHCHECK.
- In-repo: services/*/Dockerfile — plus a “broken” target that exits 78.

GET YOUR HANDS DIRTY

```
docker build -f services/orders/Dockerfile -t  
x .  
docker run --rm -p 3001:3001 x  
docker logs -f <ctr>      docker history x
```

Gate it: add a HEALTHCHECK + non-root USER; Trivy scans the image in CI.



The senior move: ship a digest, not :latest — and never build with npm install.

Docker Compose & the three tiers

Declarative multi-container topology for one host — dev, CI, and small deployments.

FOUNDATION

- Health-gated startup: `depends_on` + `condition: service_healthy`.
- Overlays: one base file + feature files (obs, security, vault).
- Named volumes for state; resource limits so nothing starves the host.
- In-repo: `docker-compose.yml` + 4 overlays, wired to the Makefile.

GET YOUR HANDS DIRTY

```
make up           # the 3 tiers
make obs          # + Prometheus/Grafana
make ps · make logs · make psql
make down / nuke
```

Gate it: validate without running: ``docker compose config -q`` in CI.



The senior move: wait for healthy, not started — and put a resource limit on every service.

Kubernetes — deploy • scale • self-heal • roll back

A declarative control loop. Its most valuable behaviour: refusing a bad deploy.

FOUNDATION

- Probes: liveness (restart) never touches the DB; readiness does.
- HPA owns replicas — never set both, or the autoscaler thrashes.
- A bad rollout STALLS: new pods never go Ready, old ones keep serving.
- In-repo: k8s/base — NetworkPolicy, PDBs, PSA, StatefulSet data tier.

GET YOUR HANDS DIRTY

```
kubectl apply -k k8s/base
kubectl -n daig get pods -w · get hpa
kubectl -n daig rollout undo deploy/orders
```

Gate it: default-deny NetworkPolicy + a PodDisruptionBudget per workload.



The senior move: let the platform refuse a bad deploy — never override the stall.

IaC — Terraform & Ansible

Describe infrastructure as reviewable, repeatable, destroyable code.

FOUNDATION

- Terraform: declarative; diff desired vs state; pin providers.
- Read the plan before every apply — plan is the review, apply is the merge.
- Ansible: idempotent — run twice, the second run is all “ok”.
- In-repo: infra/{aws,gcp,azure} — deliberately parallel; ansible/ roles.

GET YOUR HANDS DIRTY

```
terraform -chdir=infra/aws init -backend=false
terraform -chdir=infra/aws validate
ansible-playbook -i ../hosts.yml site.yml #
run 2×
```

Gate it: remote state + locking before two people touch the same infra.



The senior move: read the plan before the apply; destroy the stack at end of day.

CI/CD — the pipeline that refuses to ship broken code

Its most valuable behaviour is saying no. Actions SHA-pinned; OIDC, no long-lived keys.

The Pipeline: Prove, Gate, Ship — or Refuse

A pipeline's most valuable behaviour is refusing to deploy. Actions SHA-pinned; OIDC, no long-lived keys.



CI (prove) → progressive CD (canary → rollout → 1-step rollback) → 7 DevSecOps gates. Break a test → the build refuses to ship.

DevSecOps & Secrets

Security as gates in the pipeline, cheapest-and-fastest first — plus secrets that fail closed.

FOUNDATION

- 7 gates: secrets → SAST → SCA → IaC → image → SBOM+sign → DAST.
- Fail on high+ severity, not every low finding — or the gate gets ignored.
- Vault/OpenBao: AppRole, least-privilege policies, dynamic short-TTL creds.
- secrets.js exits 78 rather than silently falling back to env vars.

GET YOUR HANDS DIRTY

```
make scan          # full toolchain
make insecure-on   # enable planted vulns
./chaos/day6-security.sh verify
make vault-up && make vault-app
```

Gate it: sign images (cosign) + keep the SBOM; write the deny policy down.



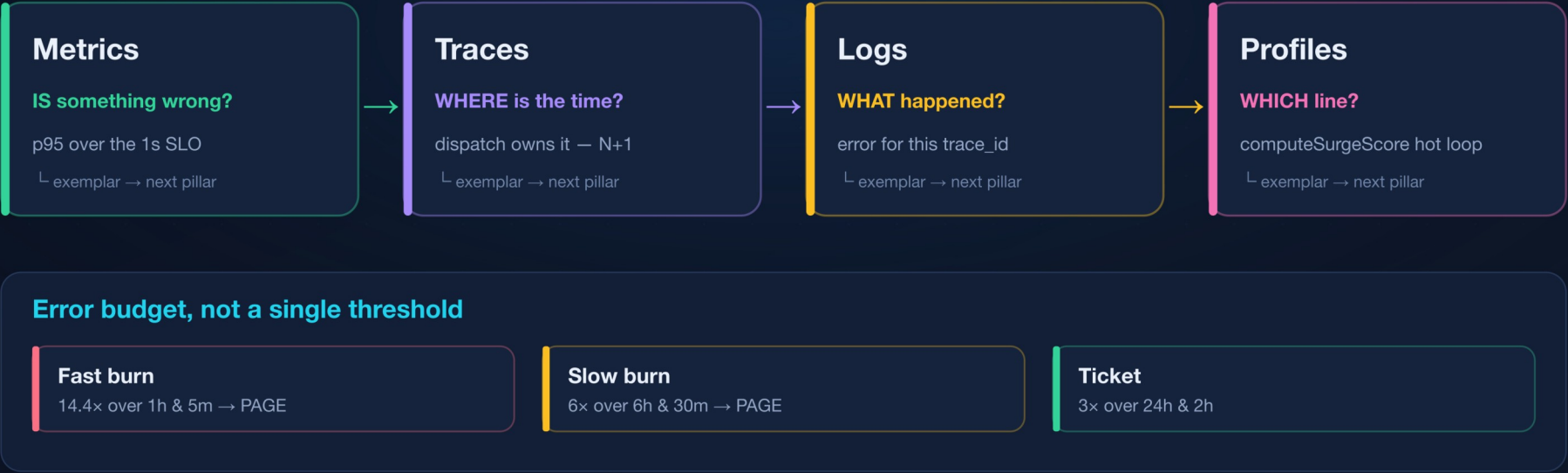
The senior move: triage the finding, then leave the rule that catches it next time.

Observability & SRE — four pillars, one request

Each pillar answers a different question. The skill is the pivot between them.

Four Pillars, One Request — the Diagnosis Pivot

Each pillar answers a different question. Alerting is symptom-based, on multi-window burn rate.



Metric (is it wrong?) → Trace (where?) → Log (what?) → Profile (which line?). Alert on symptoms, page on multi-window burn rate.

Chaos engineering — break it safely

Inject one fault, with a bounded blast radius and a one-command revert. Find weakness before it finds you.

FOUNDATION

- State a steady-state hypothesis: $p95 < 1s$, errors $< 0.1\%$.
- Inject ONE fault; observe whether steady state holds.
- If nothing lights up, your observability has a gap — that's a finding too.
- In-repo: chaos/day1...day6 — each a clear symptom + a fix/revert.

GET YOUR HANDS DIRTY

```
./chaos/day1-network.sh break    ·    ... fix  
./chaos/day3-crashloop.sh break  
./chaos/day4-latency.sh break  
./chaos/day6-security.sh break · verify
```

Gate it: the gate is the alert/policy that catches this fault next time.



The senior move: know the revert before you break — if nothing alerts, that is the bug.

The 6-day rotation

A suggested cadence. Each day = one pass around the loop at a new layer.

Day 1

Linux · Git · Containers

run the stack · read exit-78 · first image

Day 2

IaC · the cloud

terraform validate · idempotency · one cloud

Day 3

Compose → Kubernetes

tiers · deploy · scale · self-heal · rollback

Day 4

CI/CD · Observability

pipeline refuses · trace the latency spike

Day 5

DevSecOps · Secrets

7 gates · fix a vuln · Vault dynamic creds

Day 6

Chaos · Demo day

break · diagnose · gate · present the fix

How you're assessed

Evidence over assertion. Capture proof for every task in a submission/ folder.

Correctness

the fix actually resolves the symptom

Diagnosis

found from signals, not guesswork

Right layer

fixed where it belongs, not where it's easy

Gating

left a test / rule / policy / alert

Evidence

the artifact is in submission/ and legible

Communication

can explain WHY — answers the checkpoint

The rule: don't "make it green" without understanding it. daig is broken on purpose — the point is the diagnosis and the gate, not a clean terminal.

YOUR MAP FOR THE WEEK

Break it. Diagnose it. Ship it safely.



TRAINING.md

the handbook — concepts, cheatsheets, guidelines



ASSIGNMENT.md

the 7-phase capstone + acceptance checklist



SOLUTION.md

worked answers (instructor-held)



roadmap.sh/devops

the what-to-learn map — daig is the do-it companion

**Now go
make a mess.**

Clone it. Run it.
Break it. Fix it.
Gate it.

MIT • runs on a laptop